Common Mistakes on Customer Systems

1. Bad connection management
2. Bad use of cursors and the shared pool
3. Excess of resources consuming SQL statements
4. Use of nonstandard initialization parameters
5. Poor database disk configuration
6. Redo log setup problems
7. Excessive serialization
8. Inappropriate full table scans
9. Large number of recursive SQL statements related to space management or parsing activity
10. Deployment and migration errors

ORACLE

2 - 1 Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Common Mistakes on Customer Systems

1. **Bad connection management:** The application connects and disconnects for each database interaction. This problem is common with stateless middleware in application servers. It has over two orders of magnitude impact on performance, and is not scalable.
2. **Bad use of cursors and the shared pool:** Not using cursors results in repeated parses. If bind variables are not used, there may be hard parsing of all similar SQL statements. This has an order of magnitude impact on performance, and it is not scalable. Use cursors with bind variables that open the cursor and execute it many times. Be suspicious of applications generating dynamic SQL.
3. **Bad SQL:** Bad SQL is SQL that uses more resources than appropriate for the application. This can be a decision support system (DSS) query that runs for more than 24 hours or a query from an online application that takes more than a minute. SQL that consumes significant system resources should be investigated for potential improvement. ADDM identifies high-load SQL and the SQL Tuning Advisor can be used to provide recommendations for improvement.

Reasons for Inefficient SQL Performance

- Stale or missing optimizer statistics
- Missing access structures
- Suboptimal execution plan selection
- Poorly constructed SQL

ORACLE

2 - 2 Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Reasons for Inefficient SQL Performance

SQL statements can perform poorly for a variety of reasons:

- **Stale optimizer statistics:** SQL execution plans are generated by the cost-based optimizer (CBO). For CBO to effectively choose the most efficient plan, it needs accurate information on the data volume and distribution of tables and indexes referenced in the queries. Without accurate optimizer statistics, the CBO can easily be mislead and generate suboptimal execution plans.
- **Missing access structures:** Absence of access structures, such as indexes, materialized views, and partitions is a common reason for poor SQL performance. The right set of access structures can improve SQL performance by several orders of magnitude.
- **Suboptimal execution plan selection:** The CBO can sometimes select a suboptimal execution plan for a SQL statement. This happens for most part because of incorrect estimates of some attributes of that SQL statement, such as its cost, cardinality, or predicate selectivity.

- **Poorly constructed SQL:** If the SQL statement is designed poorly, there is not much that the optimizer can do to improve its performance. A missing join condition leading to a Cartesian product, or the use of more expensive SQL constructs like `UNION` in place of `UNION ALL`, are just a couple of examples of inefficient SQL design.

These four main causes of poor SQL optimization can have a drastic impact on performance.

Note: Additional reasons for poor performance might be connected with hardware-related issues, such as memory, I/Os, CPUs, and so on.

Inefficient SQL: Examples

1
```
SELECT COUNT(*) FROM products p
WHERE prod_list_price <
  1.15 * (SELECT avg(unit_cost) FROM costs c
          WHERE c.prod_id = p.prod_id)
```

2
```
SELECT * FROM job_history jh, employees e
  WHERE substr(to_char(e.employee_id),2) =
  substr(to_char(jh.employee_id),2)
```

3
```
SELECT * FROM orders WHERE order_id_char = 1205
```

4
```
SELECT * FROM employees
   WHERE to_char(salary) = :sal
```

5
```
SELECT * FROM parts_old
UNION
SELECT * FROM parts_new
```

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Inefficient SQL: Examples

The slide shows five examples of possibly poorly constructed SQL that could easily result in inefficient execution.

1. This is a common business question type. The query determines how many products have list prices less than 15% above the average cost of the product. This statement has a correlated subquery, which means that the subquery is run for every row found in the outer query. The query is better written as:

```
SELECT COUNT(*) FROM products p,
(SELECT prod_id, AVG(unit_cost) ac FROM costs
GROUP BY prod_id) c
WHERE p.prod_id = c.prod_id AND
p.prod_list_price < 1.15 * c.ac
```

2. This query applies functions to the join columns, restricting the conditions where indexes can be used. Use a simple equality, if you can. Otherwise, a function-based index may be necessary.
3. This query has a condition that forces implicit data type conversion; the `ORDER_ID_CHAR` column is a character type, and the constant is a numeric type. You should make the literal match the column type.